

ECE264-002 Spring 2020 Exam 1
8-9 PM, Wednesday, Feb 19th, 2020

1. Pointer

Consider the following program and answer question 1.A - 1.F. For each question, please only write the number in the answer sheet.

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.
4.  void f1(int* x, int* y) {
5.      int * ptr = x;
6.      x = y;
7.      y = ptr;
8.  }
9.
10. void f2(int* x, int* y) {
11.     int val = *x;
12.     *x = *y;
13.     *y = val;
14. }
15.
16. void f3(int* x, int* y) {
17.     int val = *x;
18.     x = y;
19.     y = &val;
20. }
21.
22. int main(int argc, char** argv) {
23.     int arr_a[3] = {0, 2, 4};
24.     int arr_b[3] = {1, 3, 5};
25.
26.     f1(arr_a, arr_b);
27.     // 1.A: what is the output?
28.     printf("Answer 1.A: %d\n", *arr_a);
29.     // 1.B: what is the output?
30.     printf("Answer 1.B: %d\n", *arr_b);
31.
```

```
32.    f2(arr_a + 1, arr_b + 1);
33.    // 1.C: what is the output?
34.    printf("Answer 1.C: %d\n", *(arr_a + 1));
35.    // 1.D: what is the output?
36.    printf("Answer 1.D: %d\n", *(arr_b + 1));
37.
38.    f3(arr_a + 2, arr_b + 2);
39.    // 1.E: what is the output?
40.    printf("Answer 1.E: %d\n", *(arr_a + 2));
41.    // 1.F: what is the output?
42.    printf("Answer 1.F: %d\n", *(arr_b + 2));
43.
44.    return EXIT_SUCCESS;
45. }
```

Answer:

1.A: 0

1.B: 1

1.C: 3

1.D: 2

1.E: 4

1.F: 5

2. String

Consider the following program in q2.c

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <string.h>
4.
5.  int main(int argc, char** argv) {
6.      if (argc < 4) {
7.          return EXIT_FAILURE;
8.      }
9.
10.     // 2.A: what is the output?
11.     printf("Answer 2.A: %d\n", argv[1][1]);
12.
13.     // 2.B: what is the output?
14.     printf("Answer 2.B: %ld\n", strlen(argv[2]));
15.
16.     // 2.C: please fix the following code
17.     char *arr = malloc(2.C * (strlen(argv[3]) + 1));
18.     if(arr == NULL) {
19.         return EXIT_FAILURE;
20.     }
21.     strcpy(arr, argv[3]);
22.
23.     // 2.D: what is the output?
24.     printf("Answer 2.D: %s\n", arr);
25.
26.     // 2.E: what is the output?
27.     printf("Answer 2.E: %c\n", *arr);
28.
29.     // 2.F: fix the code to print characters in arr
30.     for(int i = 0; arr[i] != 2.F; i++) {
31.         printf("%c", arr[i]);
```

```
32.    }
33.    printf("\n");
34.
35.    free(arr);
36.    return EXIT_SUCCESS;
37. }
```

Suppose we compile q2.c with the following command

```
gcc q2.c -o q2prog
```

Run the executable program with the command

```
./q2prog 264 section2 exam
```

Please write your answers to question 2.A-2.F in the answer sheet.

Answer:

2.A: 54

2.B: 8

2.C: sizeof(char)

2.D: exam

2.E: e

2.F: '\0'

3. Number Representation

Consider the following program.

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.
4.  const int THRESHOLD = 0x28;
5.
6.  int max(int x, int y) {
7.      if (x > y)
8.          return x;
9.      else
10.         return y;
11. }
12.
13. int main(int argc, char ** argv)
14. {
15.     // Hint: 0x is the prefix of hexadecimal number
16.     // Hint: 0b is the prefix of binary number
17.     int num1 = 0x39;
18.     int num2 = 0x1D;
19.     int num3 = 0b1111;
20.     int num4 = 0b1001;
21.
22.     // 3.A: what is the output?
23.     printf("Answer 3.A: %d\n", num1);
24.
25.     // 3.B: what is the output?
26.     // Don't include prefix in your answer
27.     // Hint: %o means print in octal format (base 8)
28.     printf("Answer 3.B: %o\n", num2);
29.
30.     // 3.C: what is the output?
31.     printf("Answer 3.C: %d\n", num3);
32.
```

```
33.    // 3.D: what is the output?
34.    printf("Answer 3.D: %o\n", num4);
35.
36.    int diff = max(num1 - num3, num2 - num4);
37.    // 3.E: what is the output?
38.    // Don't include prefix in your answer
39.    // Hint: %X means print in hexadecimal format
40.    // with alphabets in upper case (A-F)
41.    printf("Answer 3.E: %X\n", diff);
42.
43.    // 3.F: what is the output?
44.    printf("Answer 3.F: %X\n", max(diff, THRESHOLD));
45.
46.    return EXIT_SUCCESS;
47. }
```

Answer:

3.A: 57

3.B: 35

3.C: 15

3.D: 11

3.E: 2A

3.F: 2A

4. Call Stack

Consider the following program:

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <string.h>
4.  #pragma pack(1) // tell compiler not to pad any space
5.  // assume sizeof(char) = 1, sizeof(int) = 4,
6.  // sizeof(double) = 8, sizeof(a pointer) = 8
7.
8.  int mysquare(int num, int threshold) {
9.      if(num < threshold)
10.         return num * num;
11.     else
12.         return threshold * threshold;
13. }
14.
15. int main(int argc, char** argv) {
16.     char* nums = "51397";
17.     int sum = 0;
18.
19.     int* arr = malloc(sizeof(int) * strlen(nums));
20.     if(arr == NULL) {
21.         return EXIT_FAILURE;
22.     }
23.
24.     int threshold = nums[0]-'0';
25.     for(int iter = 0; iter < strlen(nums); iter++) {
26.         arr[iter] = mysquare(nums[iter]-'0', threshold);
27.         sum += arr[iter];
28.         printf("%d-th element: %d\n", iter, arr[iter]);
29.     }
30.     printf("%d\n", sum);
31.
32.     // 4.F: If the memory is not released, how many
33.     // bytes of memory are leaked?
```

```

34.    free(arr);
35.    return EXIT_SUCCESS;
36. }

```

Question 4.A - 4.E: please fill in the following call stack as the program runs after line 8 and before line 9. (Hint: note that function `mysquare` has been called several times in the program. Before answering the questions, please first infer at which point the program is currently at according to the call stack). In the call stack, '—' means do not worry about it.

Frame	Symbol	Address	Value	Symbol	Address	Value
mysquare	num	A201	7			
	threshold	A200	4.E			
	Value Address		4.D			
	Return Location		4.C			
main	iter	A106	4			
	threshold	A105	—			
	arr	A104	4.B	arr[4]	A10016	—
	sum	A103	4.A	arr[3]	A10012	—
	nums	A102	"51397"	arr[2]	A10008	—
	argv	A101	—	arr[1]	A10004	—
	argc	A100	—	arr[0]	A10000	—
Stack Memory				Heap Memory		

Answer:

4.A: 60

4.B: A10000

4.C: line 27

4.D: A10016

4.E: 5

4.F: 20

ASCII Table

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	NUL	32	20	SPACE	64	40	@	96	60	`
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	\$	68	44	D	100	64	d
5	5	ENQ	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL